Características fundamentales de un objeto

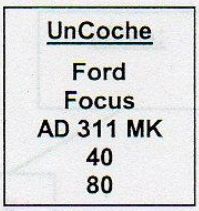
La presentación de las características fundamentales de un objeto permite responder de manera más formal a la cuestión: ¿qué es un objeto? Todo objeto presenta las tres características siguientes: un estado, un comportamiento y una identidad.

Objeto = Estado + Comportamiento + Identidad

El estado

El estado agrupa los valores instantáneos de todos los atributos de un objeto, sabiendo que un atributo es una información que cualifica al objeto que la contiene. Cada atributo puede tomar un valor en un ámbito de definición dado. El estado de un objeto, en un instante dado, corresponde a una selección de valores, entre todos los valores posibles de los diferentes atributos.

El siguiente diagrama muestra un objeto coche que contiene, en un instante dado, los valores de sus atributos: marca, patente, cantidad de combustible, temperatura.

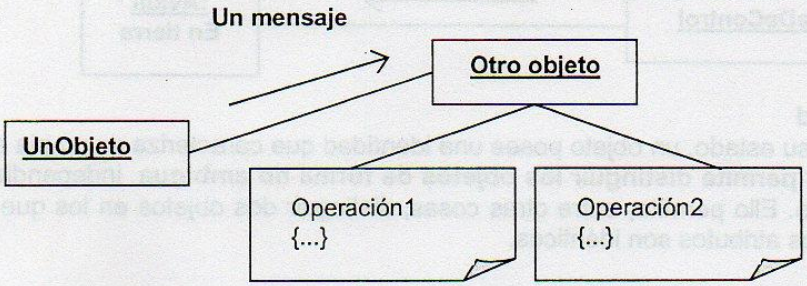


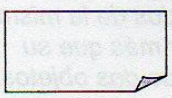
El estado evoluciona con el tiempo; así, cuando un coche circula, la cantidad de combustible disminuye y la temperatura del mismo varía. Ciertos atributos del estado pueden ser constantes, tal es el caso de la marca y el número de patente. Sin embargo, por regla general, el estado de un objeto es variable y puede verse como la consecuencia de sus comportamientos pasados.

El comportamiento

El comportamiento describe las acciones y reacciones de ese objeto.
Cada comportamiento se llama operación. Las operaciones de un objeto se desencadenan a consecuencia de un estímulo externo, representado en forma de un mensaje enviado por otro objeto.
Un **mensaje**, es una solicitud que pide al objeto que se comporte de una manera determinada.

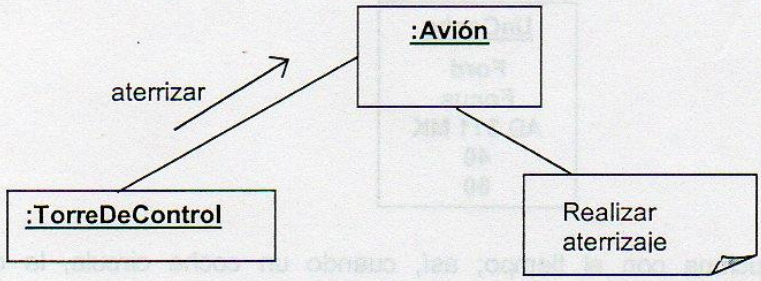
Todo mensaje está compuesto por la **identidad del receptor**, el **comportamiento a ejecutar** y opcionalmente **información adicional** para realizar el comportamiento.
En el siguiente diagrama, según el mensaje, se desencadena la *Operación1* o la *Operación2*.





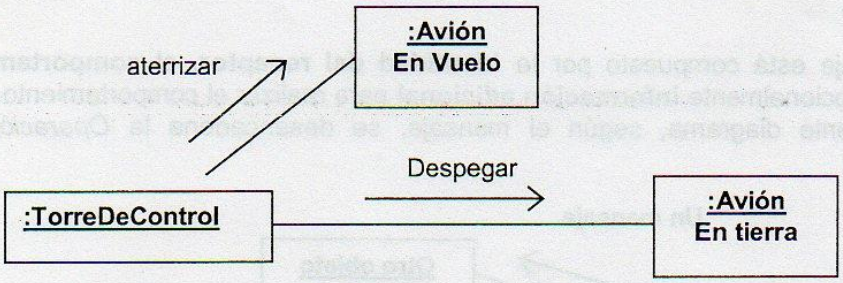
Este diagrama presenta un rectángulo cuya esquina inferior derecha está plegada: se trata de la representación de una nota, es decir, de una información de clarificación opcional expresada en un formato libre, a fin de facilitar la comprensión del diagrama.

Ejemplo:



Como veremos más adelante, las interacciones entre objetos se representan por medio de diagramas en los que los objetos que interactúan están vinculados entre sí por líneas continuas llamadas enlaces. La presencia de un enlace significa que un objeto conoce o ve a otro objeto. En el ejemplo anterior, el objeto perteneciente a la clase TorreDeControl conoce al objeto perteneciente a la clase Avión.

El **estado y el comportamiento están relacionados** ya que el comportamiento en un instante dado depende del estado actual, y el estado puede ser modificado por el comportamiento. Solo es posible hacer aterrizar un avión cuando está volando, es decir, el comportamiento *Aterrizar* sólo es válido si la información *En Vuelo* es válida. Tras el aterrizaje, la información *En Vuelo* pasa a ser no válida, y la operación *Aterrizar* deja de tener sentido. El ejemplo muestra la relación entre el estado y el comportamiento.



La identidad

Además de su estado, un objeto posee una identidad que caracteriza su propia existencia. La identidad **permite distinguir los objetos de forma no ambigua**, independientemente de su estado. Ello permite, entre otras cosas, distinguir dos objetos en los que todos los valores de los atributos son idénticos.

*El concepto de identidad se refiere al hecho de que **cada objeto es único en el mundo**, por más que su conjunto de atributos y sus valores sean exactamente iguales a los de otros objetos. Por ejemplo, dos autos del mismo modelo, color, motor, salidos de la misma línea de producción el mismo día no dejan de ser dos autos diferentes, por más que su conjunto de atributos y sus valores sean iguales. La única posibilidad de que dos objetos sean iguales es que sean el mismo objeto.*

La identidad es un concepto, no se representa de manera específica en modelado. Cada objeto posee una identidad de manera implícita.

En casos como el citado anteriormente, si es necesario diferenciar objetos, a menudo puede utilizarse un identificador procedente naturalmente del ámbito del problema. Este tipo de identificador, llamado también clave natural, puede añadirse al estado de los objetos para distinguirlos.

Por ejemplo, el DNI puede ser una clave natural para distinguir personas, el número de patente para distinguir automóviles, etc.

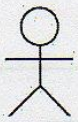
Interacciones entre objetos

Desde el enfoque del diseño orientado a objetos, una aplicación se va a modelar como un conjunto de objetos que interactúan entre sí para dar solución al pedido de un usuario. UML utiliza diagramas para modelar el sistema desde distintos aspectos, separados pero relacionados, de modo que permiten tener una visión más clara y detallada del sistema.

Una interacción expresa el comportamiento que resulta de la comunicación de un grupo de objetos. Una interacción la podemos visualizar según el punto de vista del tiempo o según el punto de vista del espacio.
Las interacciones entre objetos las podemos representar a través de dos tipos de diagramas: diagrama de comunicación y diagrama de secuencia.

Actores

La interacción de los objetos es iniciada por el usuario. Por lo que, para modelar un sistema es necesario identificar quien o quienes son sus usuarios. A partir de esto, se define el concepto de actor, **que es una entidad externa al propio sistema**, pero que necesita intercambiar información con él.
Los actores no están restringidos a ser personas físicas, también pueden representar a sistemas externos al sistema que se está modelando.
En el caso de los usuarios que son personas reales, el actor representa la función que la persona realiza, por ejemplo en un sistema de facturación de un supermercado los principales usuarios son los cajeros, por lo tanto, existirá un actor "cajero". También es probable que exista un actor "gerente de sucursal" que consulta lo facturado, "gerente general", en un contexto de una cadena de supermercados, entre otros actores.
Un actor se representa como se muestra a continuación.



Actor

4. CLASES

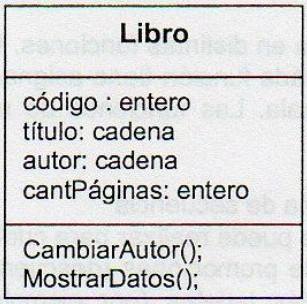
Una **clase** es una descripción de un conjunto de objetos, ya que consta de comportamientos y atributos que resumen las características comunes del conjunto.
La clase especifica el ámbito de definición de un conjunto de objetos. Cada objeto pertenece a una clase.
La posibilidad de definir clases es una de las ventajas de la orientación a objetos; definir clases significa colocar código reutilizable en un depósito común en lugar de redefinirlo cada vez que se necesite.

La **encapsulación** de una clase permite la cohesión y presenta distintas ventajas básicas:

- Se protegen los datos de accesos indebidos
- El acoplamiento entre las clases se disminuye
- Favorece la modularidad y el mantenimiento

Ejemplo: si se necesita trabajar con un conjunto de libros, para un sistema que automatiza una biblioteca, no es necesario definir el comportamiento para cada libro, será suficiente con agrupar ese comportamiento en la clase LIBRO.

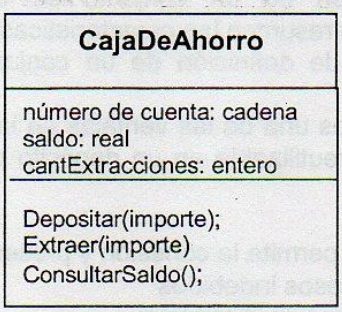
Una clase es una especificación genérica para un número arbitrario de objetos similares. Los objetos que responden a la especificación de una clase son llamados **instancias** de una clase; en nuestro ejemplo cada objeto libro es una **instancia de la clase** Libro.



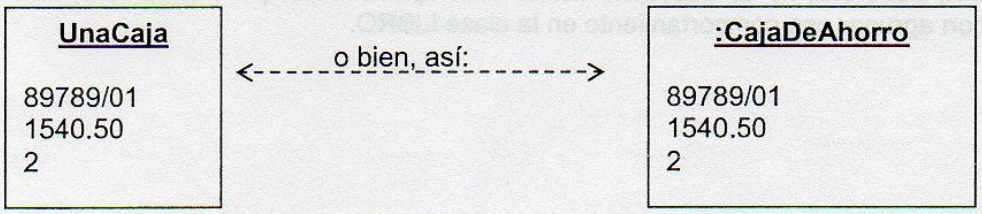
La **instanciación** es la operación que se realiza para obtener un objeto de una clase. La instanciación relaciona una clase y un objeto.

La clase tiene como atributos: código del libro, título, autor y cantidad de páginas. Los comportamientos contemplados para un objeto de esta clase son entre otros: cambiar el autor y mostrar los datos del libro.

Ejemplo: si se necesita automatizar el manejo de las cajas de ahorro de una entidad bancaria, tenemos la clase Caja de Ahorro, con las siguientes características.

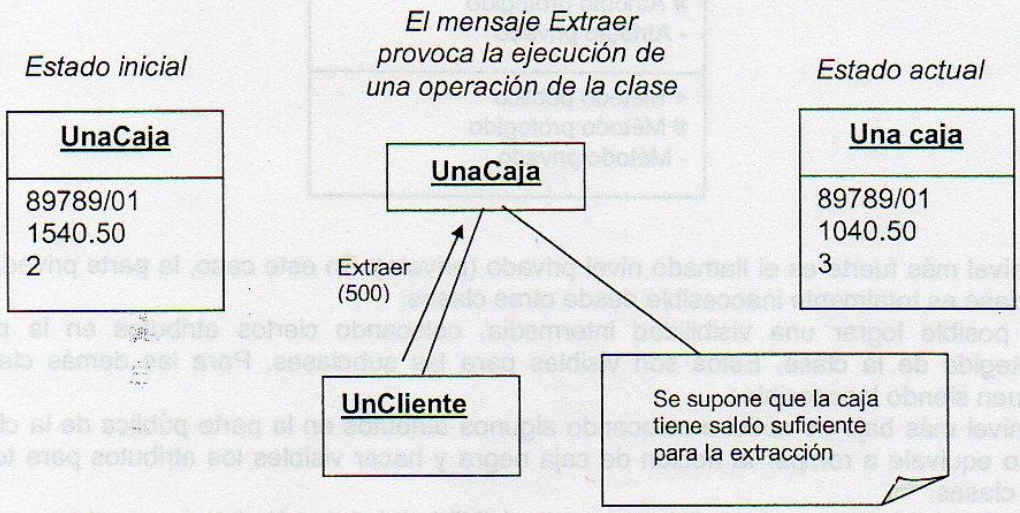


Un objeto de la clase caja de ahorro, puede representarse de la siguiente forma[12]:



[12] Como explicaremos más adelante: para distinguir entre una clase (el tipo) y un objeto (una instancia del tipo), un objeto se muestra subrayado.

El siguiente esquema muestra el cambio de estado del objeto, debido a la operación extraer.



Nótese la diferencia fundamental entre un objeto y una clase. Un objeto es algo concreto, puede ser real o abstracto. Una clase es una abstracción de un concepto que engloba todos los objetos. LIBRO es una clase mientras "Programación Orientada a Objetos – Técnicas Avanzada de Programación" es un objeto de la clase LIBRO

Visibilidad de los atributos:
Cómo antes se explicitó, los atributos de una clase no deberían ser manipulables directamente por el resto de objetos, no obstante existen distintos **niveles de encapsulación** también llamados niveles de visibilidad.

La visibilidad indica si el atributo y/o comportamiento es visible o puede ser accedido desde otras clases.
Los tres niveles distintos de visibilidad utilizados son: *private*, *protected* y *public*[13].

El nivel de visibilidad puede apreciarse en las representaciones gráficas de las clases mediante los caracteres +, # y -, que corresponden respectivamente a los niveles público, protegido y privado. Los mismos niveles de visibilidad usados para los atributos pueden aplicarse a los comportamientos.

[13] Estos niveles se corresponden con los propuestos por el lenguaje de programación C++:

Reglas de visibilidad
+ Atributo público # Atributo protegido - Atributo privado
+ Método público # Método protegido - Método privado

El nivel más fuerte es el llamado nivel privado (private). En este caso, la parte privada de la clase es totalmente inaccesible desde otras clases.
Es posible lograr una visibilidad intermedia, colocando ciertos atributos en la parte protegida de la clase. Estos son visibles para las subclases. Para las demás clases, siguen siendo inaccesibles.
El nivel más bajo se obtiene colocando algunos atributos en la parte pública de la clase. Esto equivale a romper la noción de caja negra y hacer visibles los atributos para todas las clases.
Un análisis similar puede realizarse para la visibilidad de los métodos de una clase.
El diagrama siguiente muestra la representación de la clase Caja de Ahorro.

CajaDeAhorro
-número de cuenta: cadena -saldo: real -cantExtracciones: entero
+Depositar(importe : real); +Extraer(importe: real); +ConsultarSaldo():real;

Cómo vemos, lo conveniente es que los métodos sean de acceso público y los atributos privados. De este modo se logra el **Encapsulamiento**, entendiendo por esto a la conjunción de la abstracción y el ocultamiento de la información.

5. MÉTODOS Y MENSAJES

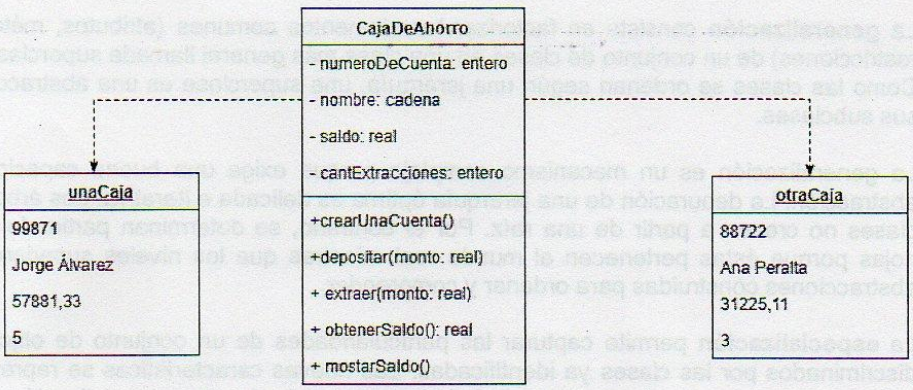
Los objetos tienen la posibilidad de actuar. La acción sucede cuando un objeto recibe un **mensaje**, que es una solicitud que pide al objeto que se comporte de manera determinada. Cada objeto recibe, interpreta y responde a mensajes enviados por otros objetos.
El conjunto de mensajes al que un objeto puede responder se llama protocolo del objeto.
Los comportamientos u operaciones que caracterizan un conjunto de objetos residen en la clase y se llaman **métodos**. En el ejemplo anterior ***ConsultarSaldo (),*** es un método de la clase **CajadeAhorro**.

Los métodos son el código que se ejecuta para responder a un mensaje, y el mensaje es la llamada o invocación a un método.

Los métodos determinan cómo actúa el objeto cuando recibe un mensaje y manipulan los valores de los atributos del objeto. De esta forma, los métodos proporcionan la única forma de modificar los datos de un objeto. La estructura de un objeto (atributos y métodos), está oculta a usuarios del objeto, de manera tal que los mensajes que recibe un objeto proporcionan la única forma de conectar al objeto con el mundo exterior. Estas características de la orientación a objetos fomentan la modularidad (ya que es clara la frontera entre los distintos objetos), explícita la comunicación entre los distintos objetos y oculta los detalles de implementación.

Representación usando UML
Para representar gráficamente clases y objetos utilizaremos la simbología propuesta por UML. En UML, una clase es representada por un rectángulo que posee tres divisiones:
Superior: Contiene el nombre de la clase.
Intermedia: Contiene los atributos que caracterizan a la clase.
Inferior: Contiene los métodos, los cuales indican la forma como interactúa el objeto con su entorno.

En UML, **un objeto** se representa bajo la forma de un rectángulo; el nombre del objeto se subraya. La primera parte del rectángulo indica el nombre y la segunda los valores asociados a los atributos.



6. HERENCIA - Conceptos de Generalización y Especialización[14]
La generalización y la especialización son puntos de vista centrados en las jerarquías de clases.

La **generalización** consiste en factorizar los elementos comunes (atributos, métodos y restricciones) de un conjunto de clases en una clase más general llamada superclase. Como las clases se ordenan según una jerarquía, una superclase es una abstracción de sus subclases.

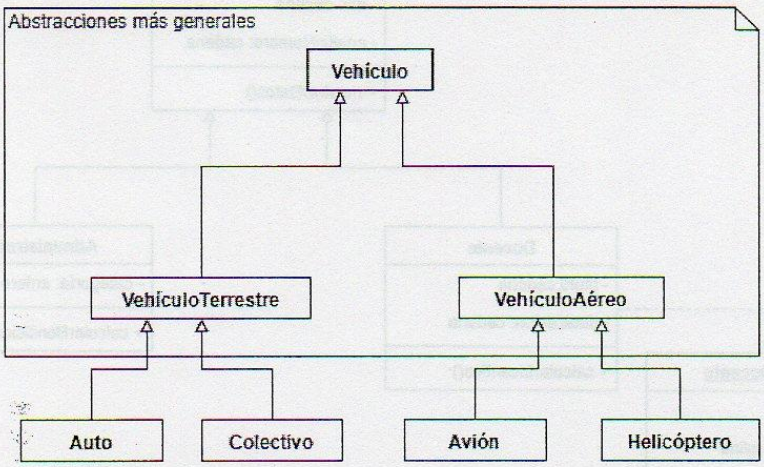
La generalización es un mecanismo complejo porque exige una buena capacidad de abstracción. La depuración de una jerarquía óptima es delicada e iterativa. Los árboles de clases no crecen a partir de una raíz. Por el contrario, se determinan partiendo de las hojas porque éstas pertenecen al mundo real mientras que los niveles superiores son abstracciones construidas para ordenar y comprender.

La **especialización** permite capturar las particularidades de un conjunto de objetos no discriminados por las clases ya identificadas. Las nuevas características se representan por una nueva clase, subclase de una de las clases existentes. La especialización es una técnica eficaz para la extensión coherente de un conjunto de clases.
Una **subclase** identifica el comportamiento de un conjunto de objetos que hereda las características de la clase padre y adicionan algunas específicas que ésta no posee.

UML utiliza la flecha  para indicar la **Especialización /Generalización**, esta indica que una subclase comparte los métodos y atributos especificados en una Clase Superior, por ende la Subclase además de poseer sus propios métodos y atributos, poseerá las características y atributos de la Clase Superior.

[14] Modelado de Objetos. op. cit. Pág.46

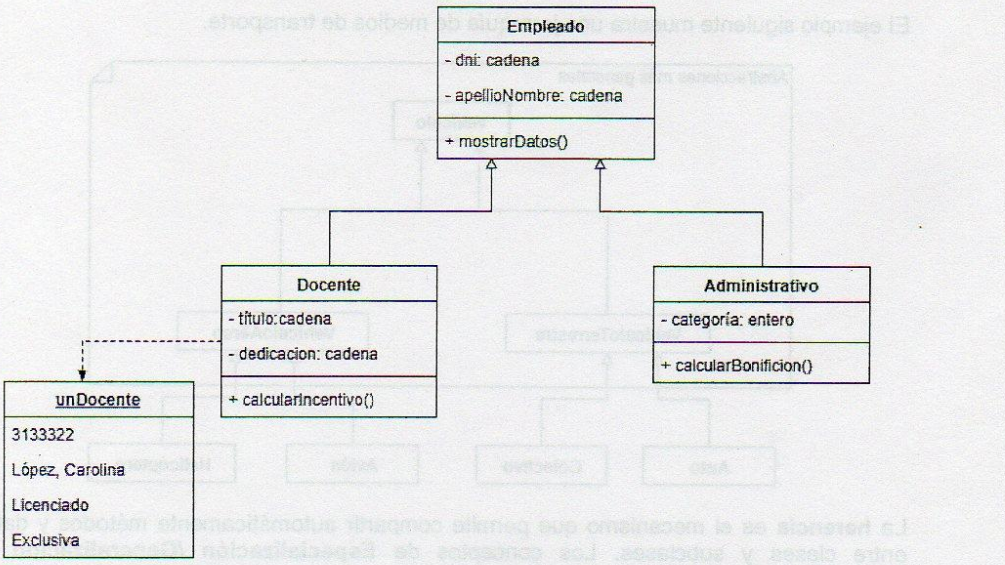
El ejemplo siguiente muestra una jerarquía de medios de transporte.



La **herencia** es el mecanismo que permite compartir automáticamente métodos y datos entre clases y subclases. Los conceptos de **Especialización /Generalización** se objetivan a través de la **herencia**.
Este mecanismo potente, permite crear nuevas clases a partir de clases existentes programando solamente diferencias.

La generación de subclases es la creación de una nueva clase que hereda de una clase existente. La única tarea de la subclase es indicar las diferencias de comportamiento y atributos entre ella y la superclase.

Ejemplo: A continuación se muestra una jerarquía de clases que forma parte de un Sistema de Administración de empleados de una universidad.



Las subclases Docente y Administrativo comparten los atributos y métodos de la clase superior Empleado. La subclase Docente por ejemplo, agrega atributos y comportamientos propios.

Clase Abstracta: Una clase es abstracta cuando no existe un objeto que sea instancia directa de ella, pero sí existe una subclase de ella que es instanciable.
Generalmente se utilizan para resumir los comportamientos comunes a un conjunto de subclases. El nombre de las clases abstractas va en cursiva. Por ejemplo, la **clase Empleado** de la jerarquía anterior es abstracta.

Clase Concreta: Una clase concreta es aquella que es instanciable, es decir que existe al menos un objeto de esa clase. La **clase Docente** es una clase concreta. El objeto **unDocente** representa una instancia de la clase Docente que además de los atributos de la clase empleado tiene los propios de la clase de la cual es instancia.
Ejemplo: Suponiendo el siguiente diagrama de clases, que responde a un Sistema de Pedido de Becas, consideremos solamente la jerarquía alumno - egresado.

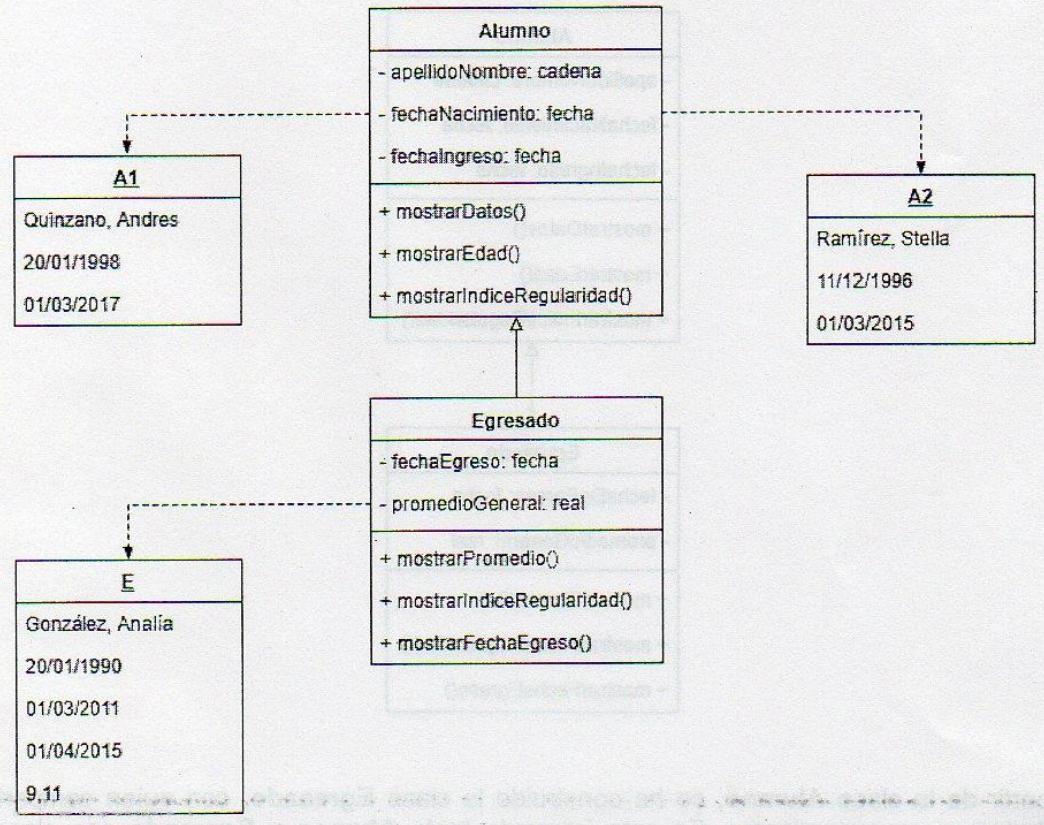


A partir de la clase **Alumno**, se ha construido la clase **Egresado**, con quien comparte atributos y comportamientos. En esta jerarquía tanto **Alumno** y **Egresado** son clases concretas, pues ambas son instanciables.

La clase **Egresado** comparte características de la clase Alumno, contiene por herencia los atributos *nombre, fechaNacimiento y fechaIngreso*, y los métodos *mostrarDatos(), mostrarEdad()*. Agrega además características particulares a través de los atributos fechaEgreso y promedioGeneral, y de los métodos mostrarPromedio(), mostrarIndiceRegularidad() y mostrarFechaEgreso().

Por otro lado el método mostrarIndiceRegularidad (), que calcula el índice de regularidad, aparece tanto en la clase superior (Alumno) como en la clase descendiente (Egresado).
Esto se debe a que el cómputo del índice de regularidad varía según la clase a la que pertenezca el objeto que recibe el mensaje. Por lo tanto, si bien el mismo método aparece en las dos clases, el método de la clase egresado redefine al de la clase alumno a través de un código distinto. Esto es, el método **mostrarIndiceRegularidad** de la clase egresado oculta al método **mostrarIndiceRegularidad** de la clase alumno.
Así, cuando un objeto de la clase egresado recibe el mensaje **mostrarIndiceRegularidad**, ejecuta el método **Egresado. mostrarIndiceRegularidad**.
Como conclusión podemos decir que, dentro de una jerarquía de clases, la herencia propaga las características de la clase superior en sus clases descendientes, de modo que varias clases pueden compartir una misma descripción.

Sean:
A1, A2: Alumno; // objetos de la clase Alumno
E: Egresado; // objeto de la clase egresado



- **A1.mostrarEdad():** este **mensaje** produce como respuesta la ejecución del **método Alumno.mostrarEdad()** definido en la clase Alumno. Este método calcula la edad a partir de la fecha de nacimiento 20/01/1998.
- **A2.mostrarEdad():** este **mensaje** produce como respuesta la ejecución del **método Alumno.mostrarEdad()** definido en la clase Alumno. En este caso el método calcula la edad a partir de la fecha de nacimiento 11/12/1996.
- **E.MostrarEdad():** en este caso el sistema busca el método **mostrarEdad** en la clase Egresado; como la clase no contiene ese método busca en la clase superior y, en respuesta al mensaje recibido, ejecuta el código asociado al **método.** En este caso el método calcula la edad a partir de la fecha de nacimiento 20/01/1990.
- **mostrarPromedio():** este **mensaje** produce como respuesta la ejecución del **método E.mostrarPromedio()** definido en la clase Egresado. Muestra el valor 9,11.

Variables de clase y variable de instancia

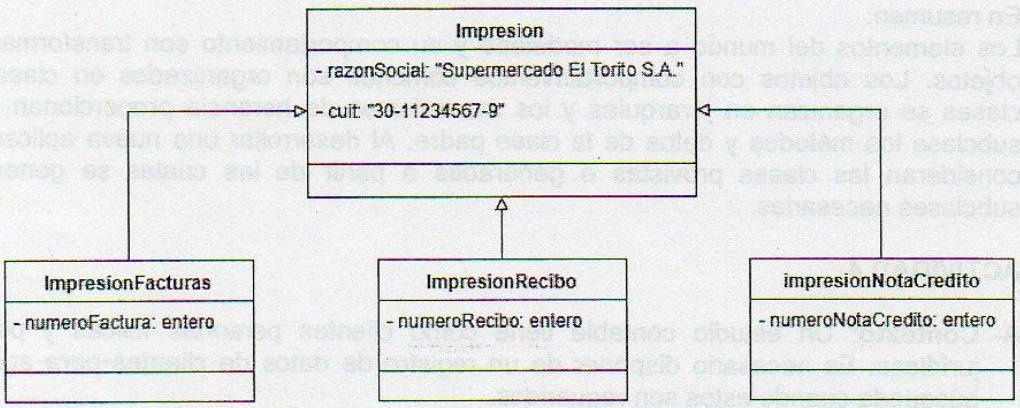
Se denominan variables o atributos de clases a aquellos atributos que tienen el mismo valor para cada objeto de la clase. A los atributos que no son de clase se los llama de instancia. Así, el valor de las variables de clase será el mismo para cualquier instancia de la clase. En una lista de objetos, una variable de clase, podría ser la cantidad de nodos, que se actualiza en dos momentos: cada vez que se crea un nodo, suma uno a la cantidad, cada vez que se elimina un nodo de la lista, se decrementa en uno la cantidad, esta variable de clase se puede utilizar para validar que se haga la limpieza correcta de la memoria utilizada por los nodos de la lista, esto es, cuando termina el programa, si la variable termina en cero, significa que se liberó toda la memoria solicitada para el almacenamiento de los nodos, caso contrario, quedan referencias a memoria que no se utiliza. Si el valor de la variable de clase es cambiado para una instancia, el mismo cambia para todas las instancias de la clase y subclase.

Una variable de clase es un elemento de información que define un atributo de toda una clase. La variable se aplica para la clase por sí misma y para todas sus instancias, de modo que sólo se almacena un valor, no importa cuántos objetos de clase hayan sido creados.

Se denominan variables o atributos de instancia a aquellos atributos cuyo valor cambiar para cada objeto de la clase.

Una variable de instancia es un elemento de información que define un atributo de un objeto en particular.
La clase del objeto define qué tipo de atributo es y cada instancia almacena su propio valor para ese atributo. A las variables de instancia también se les denomina variables de objeto.

Por ejemplo:
La siguiente jerarquía de clase modela la impresión de todo documento escrito que emita el "Supermercado EL Torito".



La jerarquía muestra una clase superior abstracta, que resume las características comunes de las subclases.
La clase **Impresión** posee al menos dos variables de clase, **razonSocial** y **cuit.** Estos atributos deben estar presentes en cualquier documento escrito de la empresa.

La clase **ImpresiónFacturas** posee al menos tres atributos, dos son variables de clase, **razonSocial** y **cuit**, y uno es de instancia que es **numeroFactura**.

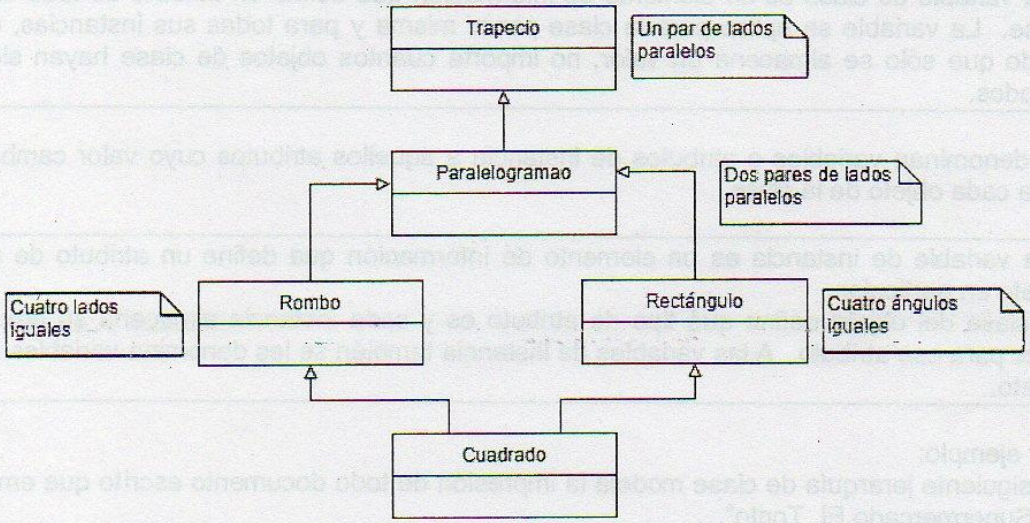
Tipos de herencia

Existen dos tipos de herencia: *herencia simple y herencia múltiple.*

Con la herencia simple una subclase puede heredar datos y métodos de una única clase y también puede añadir nuevo comportamiento. Son ejemplos de herencia simple, la jerarquía Alumno-Egresado y la jerarquía Impresión-Factura, antes presentadas.

Con la herencia múltiple una subclase puede adquirir los datos y métodos de más de una clase padre, como lo muestra la siguiente gráfica.

El siguiente ejemplo muestra, dentro de los cuadriláteros, una jerarquía que clasifica los distintos tipos de cuadriláteros trapecios.



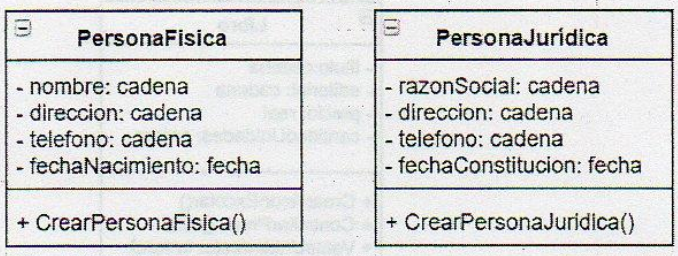
En resumen:

Los elementos del mundo a ser modelado y su comportamiento son transformados en objetos. Los objetos con comportamientos comunes son organizados en clases. Las clases se organizan en jerarquías y los mecanismos de herencia proporcionan a cada subclase los métodos y datos de la clase padre. Al desarrollar una nueva aplicación se consideran las clases provistas o generadas a partir de las cuales se generan las subclases necesarias.

ACTIVIDAD 4

A- **Contexto**: Un estudio contable tiene como clientes personas físicas y personas jurídicas. Es necesario disponer de un registro de datos de clientes para agilizar la búsqueda cuando estos son requeridos.

1. A partir de las clases dadas, aplique generalización o especialización para diseñar una jerarquía de clases adecuada para el contexto planteado.



Nota: indique que mecanismo aplicó (generalización o especialización).

2. En la jerarquía diseñada, identifique clases concretas y clases abstractas.
3. Cree una instancia de cada clase.

B- **Contexto**: Una fábrica de pastas frescas elabora los siguientes productos:

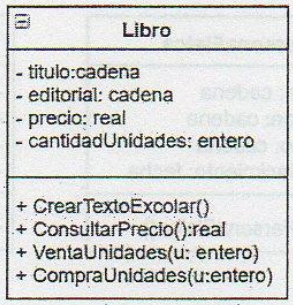
- fideos frescos: se elaboran dos tipos comunes y para celíacos. A su vez, cada tipo de fideo se presenta en caja de 500grs. y de 1000grs. El precio se establece de acuerdo con el tipo y al peso.
- ravioles: se elaboran de variedad pollo, ricota y cuatro quesos. Cualquiera sea la variedad, se presenta en caja de 500grs. y 1000grs. El precio se establece de acuerdo con el peso.
- tapas de empanadas: se presentan en paquete de 12 y 24 unidades cada uno. Se fabrican con masa criolla y masa hojaldre. El precio del paquete se establece de acuerdo con la cantidad de unidades y el tipo de masa.
- tapas de tarta: se presentan en paquete de 2 unidades cada uno, con separador y sin separador, esto última marca una distinción en el precio.

De acuerdo con el contexto planteado:

1. Identifique las características de cada tipo de producto y diseñe una clase para cada uno de ellos.
2. Aplique un mecanismo (generalización o especialización) al conjunto de clases del ítem anterior y diseñe una jerarquía de clases. Especifique el mecanismo aplicado.
3. Señale clases abstractas y concretas.
4. Represente en UML un objeto de cada clase.

C- **Contexto**: La librería "Sr Leo", vendía sólo libros de textos escolares para los distintos niveles (primario, secundario, etc.). Con el objetivo de incrementar las ventas, los dueños han decidido ampliar el rubro. A partir de este año incorpora la venta de novelas. Actualmente se usa un sistema informático que registra los libros disponibles para la venta. Los dueños han solicitado modificar el sistema actual para que permita el registro de los datos las novelas; y además registre otras características de los libros escolares (además de los datos que ya se registran), porque con el uso del sistema actual han detectado que esto necesario.

El sistema actual funciona con la siguiente clase para representar los libros de textos escolares.



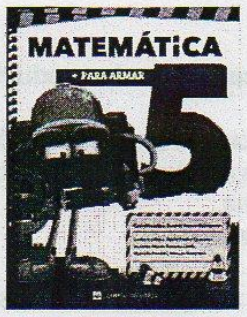
Ahora, los usuarios requieren:

1. Para el libro escolar además de los datos que ya se han registrado, se almacene el nivel (primario, secundario, etc.) al que va dirigido.
2. Para una novela registrar los mismos datos que hasta el momento el sistema registra para los libros escolares (título, editorial, precio, cantidad de unidades) en conjunto con el autor (en caso de ser más de uno, se registran los nombres separados por guion medio) y el tipo (romántica, crónica, etc.).

Para el contexto planteado:

1. Reutilizando la clase existente, diseñe una jerarquía de clases que contemple los requerimientos de los usuarios. Especifique el mecanismo (generalización o especialización) aplicado.
2. Señale clases abstractas y concretas.
3. Represente en UML los siguientes libros. Nota: de cada uno hay 10 unidades disponibles





7. RELACIONES ENTRE CLASES

En la primera etapa del DOO, se deciden las características de cada clase, es decir se identifican las clases. Una vez realizado esto, se trata de decidir el modo en que las clases se vinculan otras clases. Aunque algunas clases pueden existir aisladas, la mayoría no puede, y deben cooperar unas con otras.

Las relaciones entre las clases expresan una forma de acoplamiento entre ellas. Según el tipo de acoplamiento que presentan las clases podemos distinguir distintos tipos de relaciones. Entre ellas podemos citar:

- ✓ Asociación
- ✓ Agregación y Composición
- ✓ Herencia

Multiplicidad de las relaciones

En UML la multiplicidad indica el número de instancias (cardinalidad) que participan en la relación, y se anota en cada extremo de la relación.
Los valores de multiplicidad más comunes son:

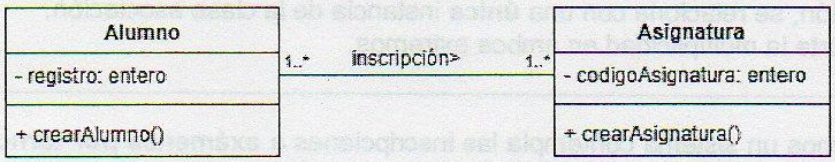
1 uno y sólo uno
0..1 cero o uno
m..n de m a n (m y n enteros naturales)
* de cero a varios
0..* de cero a varios
1..* de uno a varios

Asociación

La relación de asociación representa una conexión semántica bidireccional entre dos clases.
Una asociación es una abstracción de los vínculos que existen entre objetos instancias de las clases asociadas.
La asociación no es contenida por las clases, ni subordinada a las clases asociadas; es el reflejo de una conexión que existe en el ámbito de la aplicación.
Las asociaciones pueden ir acompañadas de un nombre que permite una mejor legibilidad del diagrama y se representa por una línea continua entre las clases asociadas.

Ejemplos:

Un alumno puede inscribirse en varias asignaturas, una asignatura permite la inscripción de varios alumnos.

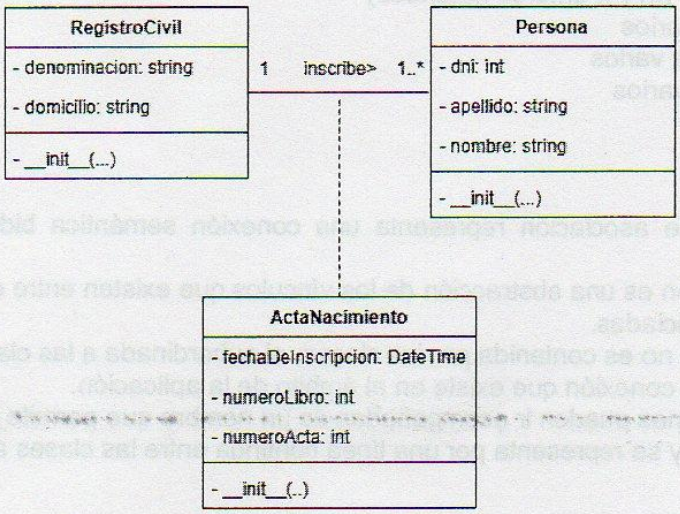


Un docente puede ser director de tesis de varios alumnos, un alumno puede tener un solo director de tesis.



Clase de Asociación

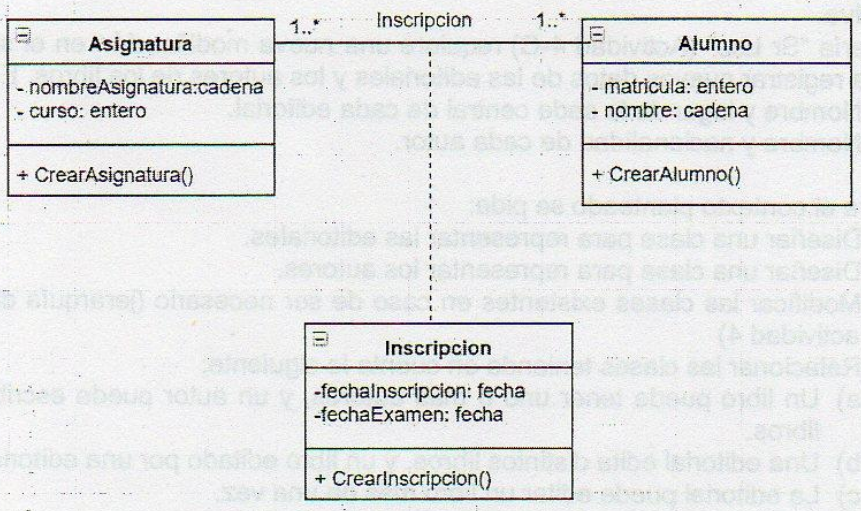
En algunas situaciones es necesario mantener la información que es específica de la propia asociación.
Supongamos un sistema que almacena información sobre los registros civiles de la provincia de San Juan, y las personas nacidas en la jurisdicción del mismo. Los nacimientos se registran en un acta de nacimiento, que posee entre otros datos: número de acta, fecha de inscripción, número de libro, etc. Como se puede observar en este ejemplo, si bien, hay una asociación entre Registro Civil y Persona, los datos propios del acta de nacimiento, no son datos inherentes al Registro Civil, ni a la Persona, esta información debe ser almacenada como atributos de una **clase asociación**.



Principio Fundamental:

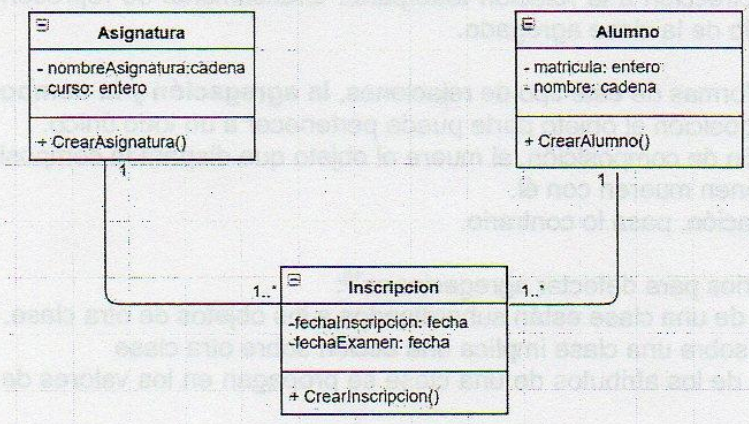
Una dupla de objetos, instancias de cada una de las clases que participan en la asociación, se relaciona con una **única** instancia de la clase asociación.
No importa la multiplicidad en ambos extremos.

Analicemos un sistema contempla las inscripciones a **exámenes por turno (por ejemplo turno del mes de julio)**, donde cada alumno puede inscribirse en una o más asignaturas, y por cada inscripción recibe un comprobante.



Para la asociación entre las clases **Asignatura** y **Alumno** de este sistema, puede crearse una clase asociación –**Inscripcion**- que guarde información de la inscripción a examen, que realiza el alumno (fecha de inscripción, fecha de examen, etc.).
Un objeto instancia de **Inscripcion** está asociado a un único y alumno y una única materia.

Lo anterior, cambia si el sistema contempla **TODAS LAS MESAS DE EXÁMENES DE JULIO** (dos turnos) donde el alumno podría inscribirse dos veces en la misma asignatura.



Para el par formado por un determinado objeto de la clase Asignatura y un determinado objeto de la clase Alumno, existe más de un objeto de la clase Inscripcion asociado a ella. En este caso no podemos definir a Inscripcion como una clase asociación, sino que existe una relación de asociación entre Asignatura y Alumno **que se modela a través de** Inscripcion.

ACTIVIDAD 5

Narrativa

La librería "Sr Leo" (Actividad 4-C) requiere una nueva modificación en el sistema que le permita registrar nuevos datos de las editoriales y los autores de los libros. Estos son:

1- Nombre y lugar de la sede central de cada editorial.
2- Nombre y nacionalidad de cada autor.

Para el contexto planteado se pide:

1- Diseñar una clase para representar las editoriales.
2- Diseñar una clase para representar los autores.
3- Modificar las clases existentes en caso de ser necesario (jerarquía diseñada en la actividad 4)
4- Relacionar las clases teniendo en cuenta lo siguiente:
   a) Un libro puede tener uno o más autores, y un autor puede escribir uno o más libros.
   b) Una editorial edita distintos libros, y un libro editado por una editorial.
   c) La editorial puede editar un libro más de una vez.
   d) De cada vez que la una editorial ha editado un libro es necesario registrar el número de edición, el año y la cantidad de ejemplares.

Agregación [15]

La forma más simple de reutilizar una clase es simplemente haciendo una **nueva clase que la contenga.** Esta técnica se llama Agregación.
Así, una agregación es una asociación *no simétrica* (todo/parte) en la que una de las clases cumple un papel predominante respecto de la otra. Por otra parte, la agregación declara una dirección a la relación todo/parte. Gráficamente se representa colocando un rombo del lado de la clase agregado.

Existen dos formas de este tipo de relaciones, **la agregación y la composición**.

• Con la composición el objeto parte puede pertenecer a un todo único.
• En la relación de composición, si muere el objeto que dispara la composición, los objetos que lo componen mueren con él.
• En la agregación, pasa lo contrario.

Algunos criterios para detectar agregaciones[16]:

- Los objetos de una clase están subordinados a los objetos de otra clase.
- Una acción sobre una clase implica una acción sobre otra clase
- Los valores de los atributos de una clase se propagan en los valores de los atributos de otra clase

En caso de duda son preferibles las asociaciones, ya que en general siempre hay que elegir una solución que implique el acoplamiento más débil.

---

[15] Resulta interesante ver los ejemplos planteados en http://www.unirioja.es/cu/jearansa/0910/archivos/EIPR_Tema02.pdf
[16] Modelado de Objetos. op. cit. Pág103

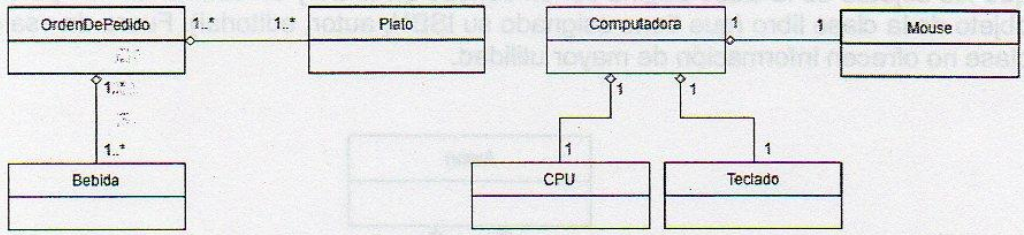
Relación de Agregación[17]

Este tipo de relación se presenta en aplicaciones en las cuales un objeto contiene como partes a objetos de otras clases., pero de tal modo que la destrucción del objeto continente no implica la destrucción de sus partes.

En este caso se dice que el objeto continente o contenedor incluye referencias a objetos de otras clases y que los objetos contenidos pueden existir independientemente del objeto contenedor. Por consiguiente los tiempos de vida de esos dos objetos no están tan estrechamente acoplados, de modo que se pueden crear y destruir instancias de cada clase independientemente.

La agregación se representa por una flecha con un rombo sin relleno

Ejemplos:



Una orden de pedido incluye uno o más objetos de Bebida, y uno o más objeto de Plato. La destrucción de un objeto Orden de Pedido, no implica la destrucción de los objetos Bebida o Plato. Análisis similar pata un objeto computadora. Los objetos de las clases CPU, teclado y pantalla tienen existencia propia independiente de la clase Computadora,.

Como se observa, no hay restricciones sobre la multiplicidad del agregado,

Relación de Composición [18]

Siguiendo con lo antes expuesto, hay agregaciones donde los objetos agregados no tienen sentido fuera del objeto resultante.

Este es el caso de la composición la cual aparece cuando los atributos de una clase hacen referencia a objetos de otra clase y éstos están físicamente contenidos por el agregado.

Así, un objeto de la nueva clase (contenedor) incluye o contiene objetos de otras clases ya definidas pero cuya existencia solo tienen sentido en este contexto.

En este caso, *la vida de los objetos incluidos depende de la vida del objeto que los incluye*. La composición se representa con un **rombo relleno** que apunta al agregado.

---

[17] En UML 1 se la denomina *agregación de contenido por referencia*

[18] En UML 1 se la denomina *Agregación de contenido por valor*

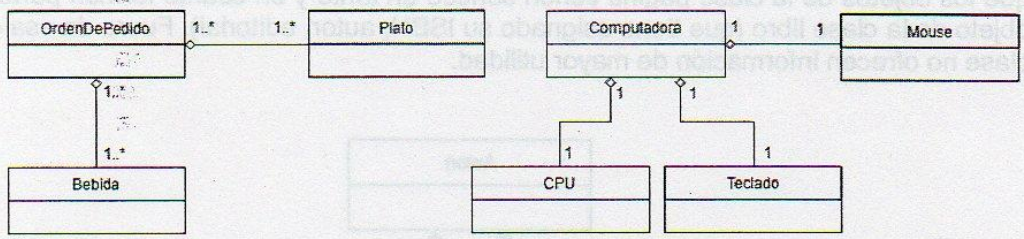
Relación de Agregación[17]

Este tipo de relación se presenta en aplicaciones en las cuales un objeto contiene como partes a objetos de otras clases., pero de tal modo que la destrucción del objeto continente no implica la destrucción de sus partes.

En este caso se dice que el objeto continente o contenedor incluye referencias a objetos de otras clases y que los objetos contenidos pueden existir independientemente del objeto contenedor. Por consiguiente los tiempos de vida de esos dos objetos no están tan estrechamente acoplados, de modo que se pueden crear y destruir instancias de cada clase independientemente.

La agregación se representa por una flecha con un rombo sin relleno

Ejemplos:



Una orden de pedido incluye uno o más objetos de Bebida, y uno o más objeto de Plato. La destrucción de un objeto Orden de Pedido, no implica la destrucción de los objetos Bebida o Plato. Análisis similar pata un objeto computadora. Los objetos de las clases CPU, teclado y pantalla tienen existencia propia independiente de la clase Computadora,.

Como se observa, no hay restricciones sobre la multiplicidad del agregado,

Relación de Composición [18]

Siguiendo con lo antes expuesto, hay agregaciones donde los objetos agregados no tienen sentido fuera del objeto resultante.

Este es el caso de la composición la cual aparece cuando los atributos de una clase hacen referencia a objetos de otra clase y éstos están físicamente contenidos por el agregado.

Así, un objeto de la nueva clase (contenedor) incluye o contiene objetos de otras clases ya definidas pero cuya existencia solo tienen sentido en este contexto.

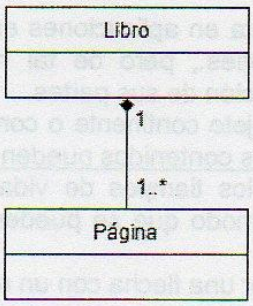
En este caso, *la vida de los objetos incluidos depende de la vida del objeto que los incluye*. La composición se representa con un **rombo relleno** que apunta al agregado.

---

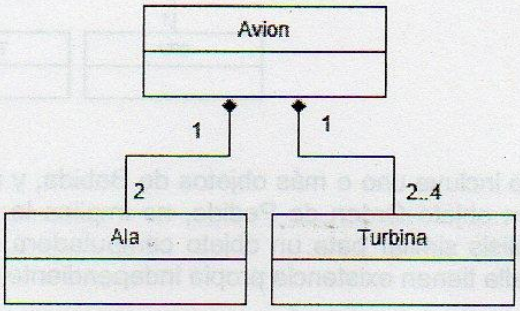
[17] En UML 1 se la denomina *agregación de contenido por referencia*

[18] En UML 1 se la denomina *Agregación de contenido por valor*

Ejemplo:



Para explicar porque es una relación de composición (agregación fuerte) cabe hacer notar que los objetos de la clase página tienen sentido en tanto y en cuanto forman parte de un objeto de la clase libro (que tiene asignado su ISBN, autor, editorial). Fuera de esa de esa clase no ofrecen información de mayor utilidad.



En este ejemplo un Avión es un objeto compuesto de otros objetos, incluye alas, turbinas, tren de aterrizaje etc. Esto significa que el objeto ala o hélice no existe independientemente de la instancia de Avión que los contiene. La vida de los objetos Avión y ala o hélice están interrelacionadas, de modo que cuando se crea una instancia de Avión se crea una instancia de la clase ala o hélice, y cuando se destruye un objeto de Avión, por implicación se destruyen también los correspondientes objetos ala o hélice.

En este caso de agregaciones, la multiplicidad del lado agregado puede tomar el valor 0 o 1. El valor 0 se refiere a un atributo no explicitado.

Reutilización y extensión[19]:
Se denomina reutilización al uso de clases u objetos desarrollados y probados en un determinado contexto, para incorporar esa funcionalidad en una aplicación diferente a la de origen. En este sentido el uso de una clase ya constituye reutilización del código.

La extensión se basa en aprovechar las clases desarrolladas para una aplicación, utilizándolas para la construcción de nuevas clases, en la misma u otra aplicación. De todas formas, como la extensión es una forma particular de reutilización –usando una

[19] http://apdaza-poo.blogspot.com.ar/2008/04/reutilizacin-del-cdigo-y-relaciones.html

acepción más amplia de término-, se suele englobar bajo el concepto común de reutilización.

Un caso típico de extensión se logra a través de la herencia, como ya se ha visto anteriormente. Así, a partir de una clase desarrollada pueden generarse una o más subclases que posean características particulares, que heredan atributos y comportamientos de la clase padre.

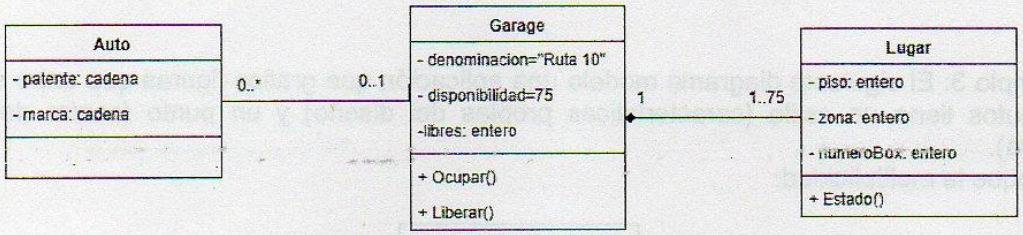
La forma más simple de reutilizar una clase es simplemente haciendo una **nueva clase que la contenga.**

Diagrama de clases

Un diagrama de clases es un diagrama estático que describe la estructura de un sistema mostrando sus clases, atributos y las relaciones entre ellos

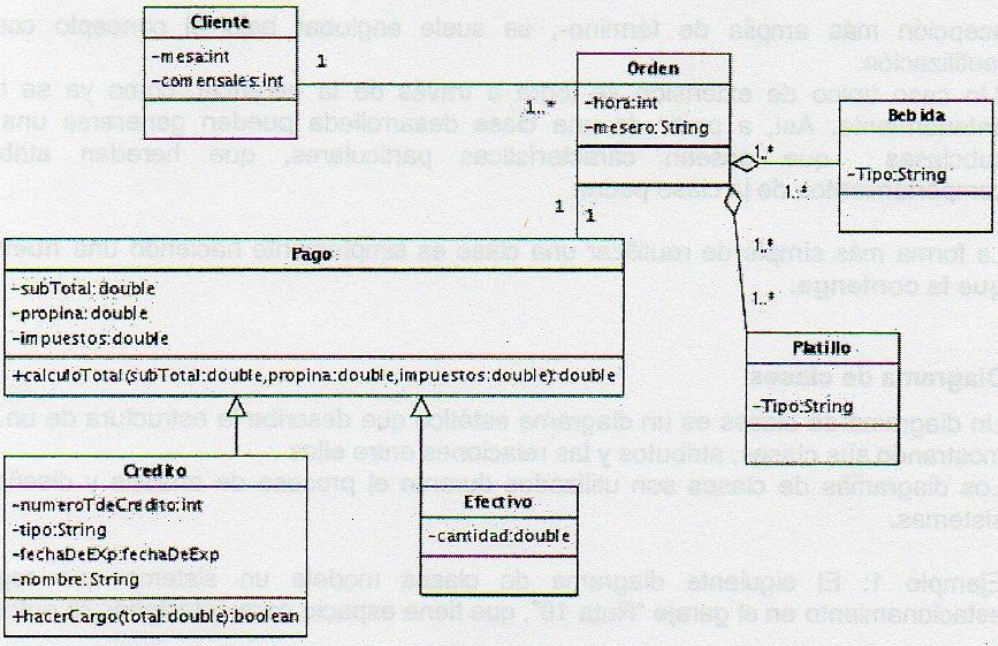
Los diagramas de clases son utilizados durante el proceso de análisis y diseño de los sistemas.

Ejemplo 1: El siguiente diagrama de clases modela un sistema que registra el estacionamiento en el garaje "Ruta 10", que tiene espacio para estacionar 75 autos.

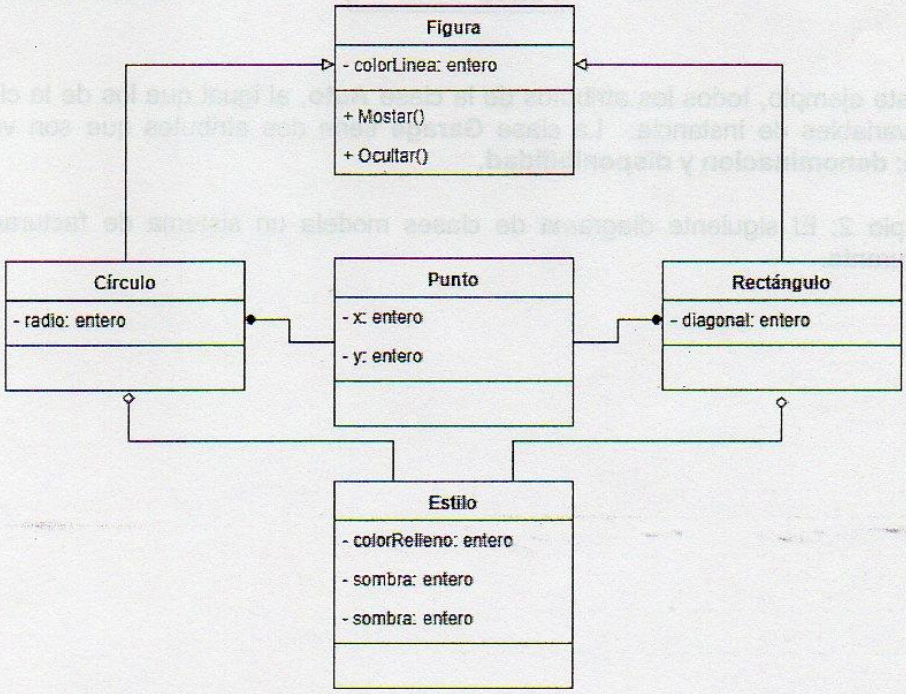


En este ejemplo, todos los atributos de la clase **Auto**, al igual que los de la clase **Lugar**, son variables de instancia. La clase **Garage** tiene dos atributos que son variables de clase: **denominacion** y **disponibilidad**.

Ejemplo 2: El siguiente diagrama de clases modela un sistema de facturación en un restaurante.



Ejemplo 3: El siguiente diagrama modelo una aplicación que grafica figuras que entre sus atributos tiene un estilo (características propias del diseño) y un punto (centro de la figura).
Coloque la multiplicidad.



ACTIVIDAD 6: Relacione las clases dadas de acuerdo con el contexto planteado.

Contexto:
El jardín maternal "Lucecita" requiere un sistema para manejar la información de los niños que asisten en el presente año.
Se reciben niños desde los 45 días hasta los 3 años de edad. De cada uno de ellos se registra los datos de al menos una persona responsable del niño (padre, madre y/o tutor). Pudiendo ser una misma persona responsable de más de un niño.
El jardín dispone de distintas salas. Un niño se asigna a una sala según la edad, y se mantiene en la misma sala durante todo el año.
La cantidad de docentes a cargo de una sala varía en función de la cantidad de niños que asisten a la sala. En un día, una docente solo puede estar a cargo de una sala, pero en el transcurso del año, una misma docente puede estar a cargo de diferentes salas. Por ello es necesario contar con un registro diario de periodo de tiempo que una docente estuvo a cargo de una sala.
El jardín cuenta con la asistencia médica de un grupo de pediatras para que controlen el estado general de salud de cada niño que se inscribe en el jardín. Cuando el médico controla a un niño completa la ficha de salud del niño. El niño es controlado solo por un médico.

El sistema requerido para manejar la información del jardín debe dar solución a los siguientes requerimientos de los dueños del jardín.

- Alta, baja, modificación para un niño, para un responsable, de una ficha de salud, de un registro diario de docente a cargo.
- Dado el DNI de un niño, mostrar las datos las personas responsables del niño.
- Dado el nombre de una sala, listar los nombres de los niños que asisten a la sala dada.
- Dada el nombre de una sala y una fecha, mostrar los nombres de las docentes que estuvieron a cargo de la sala en la fecha dada.
- Dado el DNI de un niño, mostrar los datos de la ficha de salud del niño con DNI dado y el nombre del médico que lo controló.

No es de interés resguardar los datos de los años anteriores al actual.

Para el contexto planteado se han identificado las siguientes clases:



ACTIVIDAD 7

En cada uno de los contextos presentados:

1. Identifique las clases que intervienen.
2. Diseñe el diagrama de clases (relaciones entre clases) de acuerdo con lo que el contexto planteado requiere.
3. Realice el **diagrama de secuencia** para la funcionalidad que se indica, utilizando estereotipos.
4. A partir del diagrama de secuencia construido, identifique los métodos necesarios y agréguelos en la clase correspondiente.

A- Contexto: Agencia de Turismo

La agencia de turismo "Por el mundo" que tiene sucursales en todo el país, lo ha contratado para desarrollar una aplicación que mantenga información de los paquetes turísticos ofrecidos por sus sucursales.

De cada sucursal se almacena el nombre, teléfono y dirección. Cada sucursal ofrece diferentes paquetes turísticos formados por uno o más destinos. De cada uno de los paquetes se debe almacenar un código y la duración en días. Mientras que para los destinos se necesita mantener su nombre, la provincia en la que se encuentra y el país.

Las sucursales pueden vender el mismo paquete para distintas fechas, por lo que para cada venta que una sucursal hace de un paquete se debe registrar el precio, fecha de salida y fecha de regreso.

Además, un paquete puede incluir distintos servicios. Existen dos tipos de servicios: alquiler de vehículo y excursiones. De un alquiler de vehículo se debe conocer la fecha, tipo de vehículo y cantidad máxima de pasajeros. De las excursiones se registra la fecha, actividad a realizar y vestimenta recomendada.

Restricciones:

- Un paquete se puede vender en una o varias sucursales y cada sucursal puede vender uno o varios paquetes
- Una venta tiene un paquete y una sucursal.
- Un paquete puede incluir uno o más servicios.
- Un servicio puede ofrecerse en distintos paquetes.
- Un paquete puede incluir uno o más destinos.
- Un destino puede pertenecer a varios paquetes.

Considerando el diagrama de clases elaborado en el ítem anterior y la descripción de la funcionalidad dada a continuación construya el **diagrama de secuencia**.

Funcionalidad: Un usuario selecciona la opción de "Ver paquete", la aplicación debe solicitar el código del paquete. El usuario ingresa el código del paquete y la aplicación deberá mostrar la duración, el nombre de cada destino incluido en el paquete y la fecha y tipo de cada servicio incluido en el paquete.

A- Contexto: empresa de servicio de streaming

La empresa FlixNet presta el servicio de streaming de contenido multimedia bajo demanda por Internet a los usuarios registrados. Actualmente la empresa sólo provee series.

Para mejorar la experiencia de sus usuarios, requiere una aplicación que permita llevar el registro de los capítulos que un usuario aún no termina de ver. Este registro consiste en mantener el minuto del capítulo en que se detuvo la reproducción por parte de un usuario.

No hay restricciones en cuanto a la reproducción de los capítulos, esto significa que un usuario puede haber detenido la reproducción (en distinto momento) de más de un capítulo, de la misma serie o de distintas series.
Una serie puede tener una o más temporadas. Una temporada tiene uno o más capítulos.
Toda serie tiene un género.
De cada género se registra el nombre.
De una serie se registra el nombre, el género, la cantidad de temporadas y sus temporadas.
De una temporada se registra el número, la cantidad de capítulos y sus capítulos.
De un capítulo se registra un código interno de identificación, el número, el título (si lo tiene), la duración en minutos y una breve descripción.
De un usuario registra su nombre, apellido, correo electrónico y clave de acceso.
Cuando un usuario reproduce un capítulo se registra el minuto del capítulo en que se detuvo la reproducción.

Considerando el diagrama de clases elaborado en el ítem anterior y la descripción de la funcionalidad "Ver serie" dada a continuación construya el **diagrama de secuencia**.
Funcionalidad: Un usuario selecciona la opción de "Ver serie", la aplicación debe mostrar el nombre de cada género. El usuario selecciona un género y la aplicación deberá listar, de cada serie que pertenece al género seleccionado, el nombre y la cantidad de temporadas. El usuario selecciona una serie. La aplicación muestra por cada temporada, el número de temporada y de cada uno de sus capítulos el número, el título, la duración y, en caso de tratarse de un capítulo que el usuario ha visto, el minuto en el que detuvo la reproducción. El usuario selecciona un capítulo y la aplicación muestra el mensaje de "Espere mientras se carga el capítulo seleccionado".

B- Contexto: aplicación de juego

Se requiere de una aplicación para un juego en el cual el jugador participa a través de un personaje.

El juego consiste en que el personaje supere una serie de niveles. Cada nivel se conforma por un conjunto de misiones. El nivel se ha superado cuando el personaje ha cumplido todas las misiones que lo conforman.

Cuando un personaje cumple una misión se registra el desempeño en el cumplimiento. Según este desempeño será la cantidad de dinero que gana. Cada vez que el personaje supera un nivel recibirá vidas.

Cuando un personaje cumple una misión, el desempeño logrado puede ser bueno, muy bueno o brillante. Un personaje puede cumplir una misión más de una vez; y se debe tener en cuenta que, para cada vez, que el personaje cumple una misión se registra el desempeño alcanzado y la fecha en que lo hizo.

El personaje adquiere recursos que son necesarios para cumplir misiones. Estos recursos pueden ser materiales o herramientas. De un recurso material, por ejemplo: madera, soga, etc., se registra el nombre del recurso y el precio que debe pagar el personaje para adquirirlo. De un recurso herramienta, por ejemplo: martillo, sierra, etc. se registra el nombre del recurso y la función de la herramienta (rompe piedras, corta madera, etc.). Además, cada herramienta requiere niveles superados para obtenerla. Por ejemplo, para adquirir el martillo se requiere haber superado el nivel 1 y 3. De cada recurso adquirido

por el personaje se registra la cantidad adquirida. Un personaje puede adquirir el mismo recurso más de una vez, en ese caso la cantidad adquirida se suma a la cantidad que disponía previamente.

De un nivel se registra el número de nivel y la cantidad de vidas que obtendrá el personaje cuando lo supere. De una misión se registra el nombre de la misión y la cantidad de dinero que gana el jugador para cada uno de los posibles desempeños (bueno, muy bueno y brillante).

De un personaje se registra su nombre, la imagen (nombre del archivo que contiene la imagen), cantidad de vidas disponibles y cantidad de dinero disponible.

A partir del diagrama de clases elaborado anteriormente, realice el diagrama de secuencia para la siguiente funcionalidad:

Adquirir un recurso material: El personaje solicita adquirir un material y se le muestra un listado con todos los materiales. El personaje elige uno de ellos e ingresa la cantidad a adquirir. En ese momento se le muestra el precio total (precio del material * cantidad ingresada) del material que desea adquirir y, en caso de que el dinero que el personaje dispone no sea suficiente para adquirirlo, se muestra la cantidad de dinero faltante. En cambio, si es suficiente, pide que confirme la compra. El personaje confirma la compra y se registra la cantidad adquirida del recurso y se actualiza la cantidad de dinero disponible.

8. Conceptos Claves

Polimorfismo

El polimorfismo puede definirse como la capacidad que tienen objetos de clases diferentes de una jerarquía, a responder de forma distinta a una misma llamada de un método.
La idea es que los objetos de distintas clases – derivadas de otra – puedan ser tratados de la misma manera. Esto es, se le apliquen los mismos mensajes aunque las implementaciones particulares de los métodos sean diferentes. Estos métodos tienen la misma semántica y distinta implementación.
Para esto se deberá trabajar con vinculación tardía o vinculación en tiempo de ejecución. Esto se aborda en la unidad 3.

El polimorfismo fomenta la extensibilidad, software escrito para invocar comportamiento polimórfico se escribe en forma independiente del tipo de los objetos a los cuales los mensajes son enviados. Por lo tanto, nuevos tipos de objetos, que pudieran responder a mensajes existentes, pueden ser agregados en dicho sistema sin modificar el sistema base.

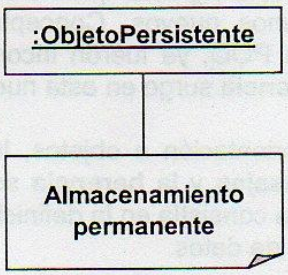
Encapsulamiento

Término formal que describe al conjunto de métodos y de datos de un objeto de manera tal que el acceso a los datos se permite solamente a través de los métodos propios del objeto.
La comunicación entre los distintos objetos se realiza solamente a través de mensajes explícitos.

Abstracción
La orientación a objetos fomenta que los programadores y usuarios piensen en las aplicaciones en términos abstractos.
A partir de un conjunto de objetos, se piensa en los comportamientos comunes de los mismos para situarlos en superclases, las cuales constituyen un depósito para elementos comunes y reutilizables.

Persistencia
La persistencia se refiere a la permanencia de un objeto, es decir, al tiempo en que se le asigna espacio y permanece accesible. Cuando un objeto ya no es necesario, es destruido, y el espacio que este ocupaba es recuperado. La recuperación automática del espacio de memoria se denomina recolección de basura.
Algunos autores estudian la posibilidad de que haya una continuidad de la existencia de un objeto luego de la ejecución de un programa, por ejemplo almacenándolo en una base de datos. Si bien hoy existen Bases de Datos Orientadas a Objetos, en la práctica estas bases de datos se basan en otros modelos como el relacional.

La persistencia de los objetos[20]
La persistencia designa la capacidad de un objeto de trascender el tiempo o el espacio. Un objeto persistente conserva su estado en un sistema de almacenamiento permanente, de modo que es posible parar el proceso que lo ha creado sin perder la información representada por el objeto (pasivación del objeto). En adelante, el objeto puede ser reconstruido (activación del objeto) por otro proceso y se comportará exactamente como en el proceso inicial. Los objetos no persistentes se llaman transitorios o efímeros. De modo predeterminado, los objetos no se consideran como persistentes.



En conjunto, los lenguajes orientados a objetos no proponen soporte directo para asegurar la persistencia de los objetos. Es una lástima, y ello obliga a recurrir a artificios externos para asegurar la persistencia de los objetos. Los fabricantes de bases de datos proporcionan soluciones para guardar objetos, bien totalmente orientadas a objetos, o bien híbridas.

Ventajas del Diseño Orientado a Objetos
Un lenguaje de programación que soporta el paradigma de orientación a objetos beneficia al desarrollador de software proporcionando una forma natural de modelar un fenómeno

[20] Extraído del libro Método de Objetos con UML

del complejo mundo real. Programar no significa solamente escribir líneas de código, sino desarrollar modelos utilizando clases.
Con la programación orientada a objetos los programas tienen menos líneas de código.
Las bibliotecas de clases predefinidas que contienen los lenguajes maduros como el Smalltalk aumentan las ventajas de utilizar lenguajes orientados a objetos, ya que la tarea de programar consiste en gran parte en encontrar dentro de las bibliotecas las clases y métodos adecuados y combinarlos convenientemente.
Otra ventaja de la programación orientada a objetos la representa la herencia, ya que subclases pueden heredar o redefinir estructuras de datos y métodos de clases existentes. Los objetos posibilitan integrar los datos y los métodos que actúan sobre dichos datos, lo que simplifica el mantenimiento del programa y sus posibles actualizaciones. Por el contrario en los lenguajes tradicionales siempre existe la posibilidad de que una actualización de una subrutina provoque efectos no deseados en subrutinas relacionadas.
Un problema potencial de los lenguajes orientados a objetos es que el programador debe conocer una extensa biblioteca de clases, pero este tiempo está compensado con el tiempo ahorrado en la reutilización de código, en lugar de tener que volverlo a crear.

Booch define la diferencia entre programar en forma tradicional (programación procedimental) y programar con objetos (programación orientada a objetos), de la siguiente manera:
"Lea las especificaciones del software que desea desarrollar. Subraye los verbos si desea código procedimental o los nombres (sustantivos), si desea código orientado a objetos."

Conclusiones
La programación orientada a objetos es producto de un proceso evolutivo permanente de los lenguajes de programación, esto significa que no se descartan viejos conceptos sino que se van incorporando algunos nuevos. Conceptos como los de **abstracción** y **encapsulamiento**, pilares de la POO, ya fueron incorporados por el **Diseño Modular**, mientras que el concepto de **herencia** surge en esta nueva metodología.

En este nuevo paradigma de orientación a objetos, los **objetos** y las **clases** son los pilares y los **métodos**, los **mensajes** y la **herencia** son mecanismos primarios. Hasta ahora la creación de un programa consistía en la definición de procesos que actuaban sobre un conjunto independiente de datos.

En la POO los **objetos** son módulos autocontenidos que hacen referencia tanto a los datos como a los procedimientos que actúan sobre dichos datos. Los procedimientos de un objeto se llaman ahora métodos y representan el comportamiento del mismo. Las características comunes de un conjunto de objetos que tienen un mismo comportamiento se agrupan en una clase, a partir de ella pueden crearse nuevas clases (subclases) que heredan los métodos y los atributos de la clase superior, permitiéndole al programador programar solamente las diferencias.